Mobile Programming

PEKAN 4 — DATA MODELING + RESTFUL API

Tema utama

FROM LOCAL APP TO CONNECTED MOBILE APP

Menghubungkan PresidentKu dengan sumber data melalui RESTful API.



Alur besar

- **PRESIDENTKU** — Aplikasi mobile
- **DATA MODEL** — Struktur data aplikasi
- **REST API** — Sumber data dari server
- **HTTP REQUEST** — Mengirim permintaan data
- **JSON RESPONSE** — Menerima data dalam format JSON
- **REACT NATIVE** — Mengolah dan menampilkan data
- **MOBILE UI** — Data ditampilkan di aplikasi

Mobile Programming

CLIENT–SERVER ARCHITECTURE

Memahami bagaimana aplikasi mobile berkomunikasi dengan server melalui RESTful API.

CLIENT vs SERVER

Mobile App (Client)
Mengirim permintaan data dan menampilkan hasilnya.

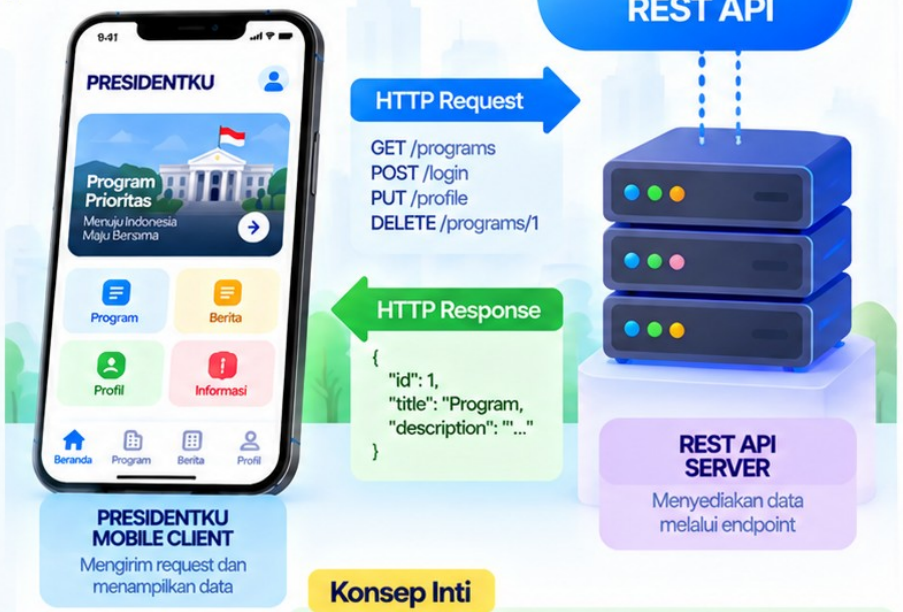
Server (Data Provider)
Menyimpan dan mengelola data aplikasi.

API (Penghubung)
Menyediakan endpoint untuk akses data.

Request
Permintaan data dari client ke server (GET, POST, dll).

Response
Hasil data dari server ke client (JSON).

Endpoint
URL spesifik untuk mengakses resource data.



Konsep Inti

Mobile App tidak harus menyimpan semua data sendiri.
Data dapat diambil dari server melalui RESTful API sehingga aplikasi lebih ringan, fleksibel, dan selalu terbarui.

Alur Komunikasi

PRESIDENTKU MOBILE CLIENT
Mengirim permintaan data

↓

HTTP REQUEST
GET /programs
POST /login, dll

↓

REST API SERVER
Memproses permintaan dan mengakses data

↓

HTTP RESPONSE
Mengirim data dalam format JSON

↓

PRESIDENTKU MOBILE UI
Menampilkan data di aplikasi

Mobile Programming

RESTful API

Antarmuka yang memungkinkan aplikasi mobile berkomunikasi dengan server menggunakan standar HTTP.

Konsep Utama

REST
Gaya arsitektur untuk membangun layanan web yang sederhana dan terstruktur.

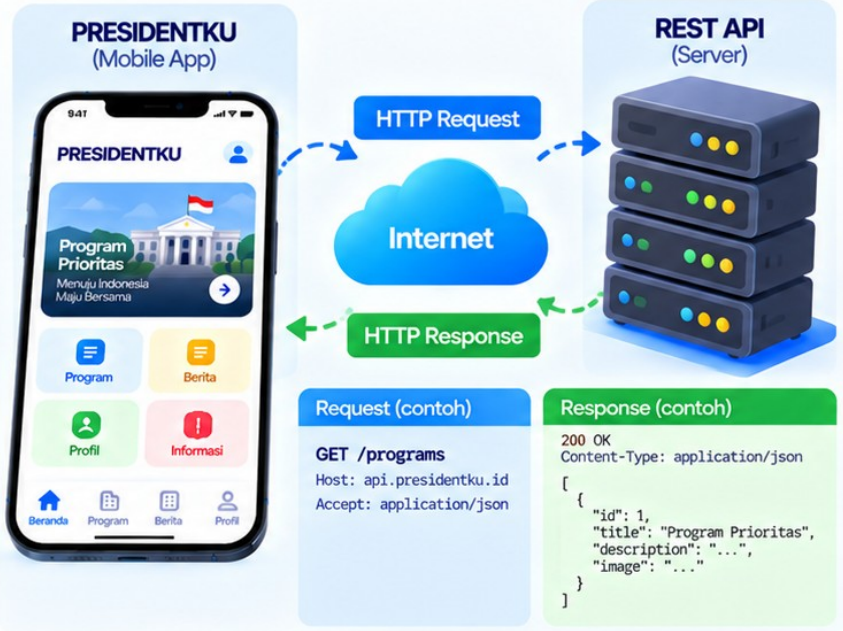
API
Antarmuka yang menyediakan akses data dan fungsi dari server.

Endpoint
URL spesifik untuk mengakses resource tertentu.

Resource
Data atau entitas yang disediakan oleh API (contoh: /programs, /berita).

HTTP Method
Metode untuk melakukan operasi pada resource (GET, POST, PUT, DELETE).

Request & Response
Permintaan data dari client dan balasan data dari server (format JSON).



HTTP Method

GET — Mengambil data
Contoh:
GET /programs
GET /berita

POST — Membuat data
Contoh:
POST /programs
POST /berita

PUT / PATCH — Mengubah data
Contoh:
PUT /programs/1
PATCH /berita/1

DELETE — Menghapus data
Contoh:
DELETE /programs/1
DELETE /berita/1

Fokus Pekan 4

Bagaimana mobile app berkomunikasi dengan API?

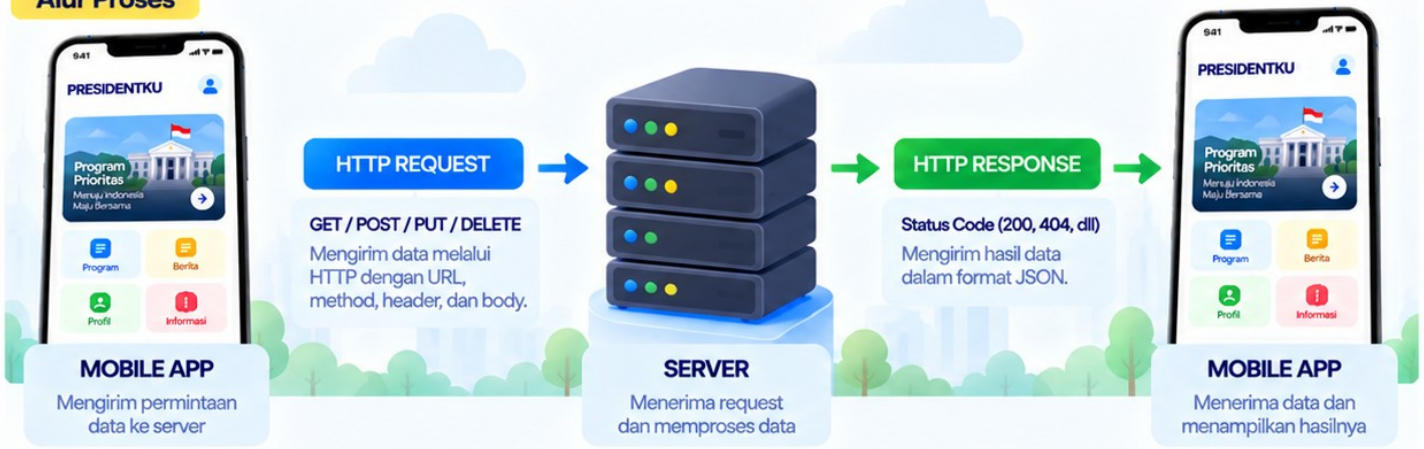
Kita akan mempelajari alur request dan response menggunakan HTTP method tanpa membahas security token secara mendalam.

Mobile Programming

HTTP REQUEST & RESPONSE

Memahami alur pertukaran data antara mobile app dan server menggunakan HTTP.

Alur Proses



Struktur HTTP Request

Request
Data yang dikirim dari mobile app ke server.

URL	Alamat endpoint API (contoh: https://api.example.com/programs)
Method	Metode HTTP (GET, POST, PUT, DELETE)
Header	Informasi tambahan (contoh: Content-Type, Authorization)
Body	Data yang dikirim (biasanya format JSON, digunakan pada POST, PUT, PATCH)

Struktur HTTP Response

Response
Data yang dikembalikan dari server ke mobile app.

Status code	Kode status hasil request (contoh: 200, 404, 500)
Header	Informasi tambahan (contoh: Content-Type: application/json)
Body	Data hasil response (biasanya format JSON)

Contoh Request & Response

HTTP Request

```
GET /programs
GET /programs HTTP/1.1
Host: api.presidenku.id
Accept: application/json

(body kosong untuk GET)
```

HTTP Response

```
200 OK
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 1,
  "name": "Program Prioritas"
}
```

Alur Sederhana

MOBILE APP
Aplikasi mengirim permintaan data.

↓

HTTP REQUEST
Berisi URL, method, header, dan body.

↓

SERVER
Memproses permintaan data.

↓

PROCESS
Mengambil atau mengubah data sesuai request.

↓

HTTP RESPONSE
Mengirim hasil dalam format JSON.

↓

MOBILE APP
Data diterima dan ditampilkan di aplikasi.

Mobile Programming

JSON DATA

Format pertukaran data yang umum digunakan pada RESTful API. Aplikasi mobile menerima data dalam format JSON, kemudian diparsing menjadi objek JavaScript untuk ditampilkan di UI.

JSON OBJECT

program.json

```
{
  "id": 1,
  "name": "Program Prioritas",
  "status": "active"
}
```

JSON ARRAY

programs.json

```
[
  {
    "id": 1,
    "name": "Program A"
  },
  {
    "id": 2,
    "name": "Program B"
  }
]
```

Alur Pemrosesan JSON

JSON
Data dalam format JSON dari server.

PARSE
Mengubah JSON menjadi objek JavaScript.

JAVASCRIPT OBJECT
Data menjadi objek yang dapat digunakan.

RENDER
Mengolah data ke komponen React Native.

MOBILE UI
Data ditampilkan di aplikasi mobile.

JSON dari API

```
{
  "id": 1,
  "name": "Program Prioritas",
  "status": "active",
  "description": "..."
}
```

JSON.parse()

JavaScript Object

```
{
  id: 1,
  name: "Program Prioritas",
  status: "active",
  description: "..."
}
```

Hasil di Mobile UI

Data JSON yang sudah diparsing ditampilkan dalam komponen UI.



Mobile Programming

DATA MODELING / DTO

Menentukan dan memetakan data dari API agar sesuai dengan kebutuhan aplikasi mobile.

Pertanyaan utama

Data dari API seperti apa yang dibutuhkan oleh aplikasi?

SERVER DATA

Data lengkap dari API (JSON).

response.json

```
{
  "id": 1,
  "name": "Program Prioritas",
  "description": "...",
  "created_at": "2024-01-01",
  "updated_at": "2024-01-10"
}
```

Data dari server biasanya lebih lengkap dari yang dibutuhkan aplikasi.

DTO / DATA MODEL

Memilih field yang dibutuhkan dan membentuk struktur data.

ProgramDTO.js

```
{
  "id": 1,
  "name": "Program Prioritas",
  "description": "..."
}
```

DTO (Data Transfer Object)
hanya berisi field yang dibutuhkan oleh aplikasi.

APPLICATION DATA

Data yang digunakan dalam aplikasi (JavaScript object).

program.js

```
{
  id: 1,
  name: "Program Prioritas",
  description: "..."
}
```

Data sudah dipetakan dan siap digunakan pada state atau komponen aplikasi.

MOBILE UI

Menampilkan data pada aplikasi PresidentKu.



Alur Data

SERVER DATA
Data lengkap dari API

→ **DTO / DATA MODEL**
Memilih field yang dibutuhkan

→ **APPLICATION DATA**
Data siap digunakan

→ **UI**
Menampilkan data di aplikasi

Konsep yang Dipelajari

- **Field** — Properti data
- **Data type** — String, number, boolean, dll
- **Object** — Struktur data dengan key-value
- **Array** — Kumpulan data
- **DTO** — Data Transfer Object
- **Mapping data** — Memetakan field dari API ke aplikasi
- **Data transformation** — Mengubah bentuk data sesuai kebutuhan

Mobile Programming

CODE EXAMPLE FETCH API

Mengambil data dari RESTful API menggunakan Fetch API pada React Native (Expo Go).

1 Fetch API (Bawaan JavaScript)

JavaScript

```
// Mengambil data menggunakan Fetch API
const response = await fetch(
    "https://api.example.com/programs"
);

const data = await response.json();

console.log(data);
```

Fetch API sudah tersedia secara default pada React Native, sehingga tidak perlu install library tambahan.

2 Axios (Library Tambahan)

JavaScript

```
// Mengambil data menggunakan Axios
const response = await axios.get(
    "https://api.example.com/programs"
);

const data = response.data;

console.log(data);
```

Axios adalah library populer yang lebih mudah digunakan dan memiliki fitur tambahan.

```
# Install Axios
npm install axios
```



1 FETCH

fetch(url)
Mengirim request ke API

2 HTTP REQUEST

GET /programs
URL: https://api.example.com/programs
Method: GET
Header: (opsional)
Body: (kosong untuk GET)

3 SERVER
Server memproses permintaan data.

4 JSON RESPONSE

```
[
  {
    "id": 1,
    "name": "Program Prioritas",
    "description": "..."
  }
]
```

5 response.json()
Mengubah response menjadi objek JavaScript.

6 DATA
Data siap digunakan di aplikasi.

Alur Singkat

1. **FETCH** — Memanggil fetch(url)
2. **HTTP REQUEST** — Mengirim request ke server
3. **SERVER** — Memproses request
4. **JSON RESPONSE** — Mengirim data dalam format JSON
5. **response.json()** — Mengubah ke objek JavaScript
6. **DATA** — Data digunakan untuk ditampilkan di UI

Mobile Programming

CODE EXAMPLE
MAPPING API DATA TO UI

Mengubah data dari API menjadi format yang sesuai untuk digunakan di aplikasi mobile.

Penting

API response tidak harus langsung dilempar ke UI. Data sebaiknya dipetakan (mapping) terlebih dahulu sesuai kebutuhan aplikasi.

1. API RESPONSE

Data yang diterima dari server (format JSON).

response.json

```
[
  {
    "id": 1,
    "name": "Program Prioritas",
    "description": "...",
    "created_at": "...",
    "updated_at": "..."
  },
  {
    "id": 2,
    "name": "Program Kedua",
    "description": "..."
  }
]
```

2. MAP (Transform Data)

Memilih field yang dibutuhkan dan mengubah struktur data.

JavaScript

```
const programs = data.map(item => ({
  id: item.id,
  title: item.name,
  description: item.description
}));

// hasilnya adalah array data
// dengan field yang lebih sederhana
```

Mapping Data

Mengambil field tertentu, mengubah nama field, atau memformat data sesuai kebutuhan UI.

3. DATA MODEL

Data dalam format yang digunakan di aplikasi (JavaScript object).

programs.js

```
[
  {
    id: 1,
    title: "Program Prioritas",
    description: "..."
  },
  {
    id: 2,
    title: "Program Kedua",
    description: "..."
  }
]
```

4. MOBILE UI

Data ditampilkan pada komponen React Native.



Alur Mapping Data

API RESPONSE — Data lengkap dari server

→ **MAP** — Memilih dan mengubah data

→ **DATA MODEL** — Format data untuk aplikasi

→ **MOBILE UI** — Menampilkan data di React Native

Field yang Digunakan

- ✓ **id** (number)
- ✓ **title / name** (string)
- ✓ **description** (string)

Field yang Tidak Digunakan dari API (contoh)

- ✗ created_at
- ✗ updated_at
- ✗ field lain yang tidak dibutuhkan

Mobile Programming

TASK 01
MODEL THE PRESIDENTKU DATA

Menentukan data apa yang diperlukan oleh aplikasi PresidentKu.

Tujuan

Mahasiswa menentukan data apa yang diperlukan PresidentKu.



Contoh Kebutuhan Data

Program

- id
- title
- description
- image
- status

Contoh JSON Structure

program.json

```
{
  "id": 1,
  "title": "Program Prioritas",
  "description": "Program strategis untuk meningkatkan kualitas pendidikan ...",
  "image": "https://.../program.jpg",
  "status": "active"
}
```

Output

PRESIDENTKU DATA MODEL

Program

id	number
title	string
description	string
image	string
status	string (active/inactive)

Task

1. **Menentukan resource aplikasi** — Contoh: Program, Berita, Profile, dll.
2. **Menentukan field** — Field apa saja yang dibutuhkan.
3. **Menentukan tipe data** — String, number, boolean, dll.
4. **Menentukan JSON structure** — Bentuk data sesuai dengan API.
5. **Membuat data model** — Struktur data yang digunakan di aplikasi.

Mobile Programming

TASK 02
CONNECT PRESIDENTKU TO REST API

Mengambil data dari API dan menampilkannya pada mobile UI.

Tujuan

Mengambil data dari API dan menampilkannya pada aplikasi PresidentKu.

Alur Proses

- **PRESIDENTKU** — Mobile App
- **REQUEST** — Mengirim request ke API
- **REST API** — Server memproses request
- **JSON** — Mengirim data dalam format JSON
- **PARSE** — Mengubah JSON menjadi objek JavaScript
- **DATA MODEL** — Memetakan data sesuai kebutuhan aplikasi
- **UI** — Menampilkan data pada mobile UI

Contoh API Request (Fetch)

```javascript
// Mengambil data dari API
const response = await fetch("https://api.example.com/programs");
const data = await response.json();

// Mapping data sesuai kebutuhan aplikasi
const programs = data.map(item => ({
  id: item.id,
  title: item.name,
  description: item.description,
  image: item.image,
  status: item.status
}));

// Simpan ke state
setPrograms(programs);
```

Contoh API Response (JSON)

```json
[
  {
    "id": 1,
    "name": "Program Prioritas",
    "description": "Program strategis ....",
    "image": "https://.../program.jpg",
    "status": "active"
  },
  {
    "id": 2,
    "name": "Program Kedua",
    "description": "Program untuk ...",
    "image": "https://.../program2.jpg",
    "status": "inactive"
  }
]
```

Hasil Mapping (Data Model)

```javascript
const programs = data.map(item => ({
  id: item.id,
  title: item.name,
  description: item.description,
  image: item.image,
  status: item.status
}));
```

Data Model

Hanya field yang dibutuhkan oleh aplikasi.

Sebelum (Static Data)



Static Data

Data masih disimpan di dalam aplikasi (secara manual).

Sesudah (Dynamic Data)



REST API

Data diambil dari server secara real-time (dynamic data).

Mobile Programming

TASK 03
DISPLAY API DATA

Mengintegrasikan data dari API ke dalam interface PresidentKu yang sudah dibuat pada Pekan 2.

Tujuan

Menampilkan data dari API pada UI PresidentKu yang sudah dibuat sebelumnya (Pekan 2).

1 UI Pekan 2 (Static Data)

Interface PresidentKu yang sudah dibuat sebelumnya menggunakan data statis (dummy).



2 Mengambil Data dari API

Menggunakan Fetch atau Axios untuk mengambil data program dari REST API.

```javascript
// Mengambil data program dari API
const response = await fetch(
  "https://api.example.com/programs"
);

const data = await response.json();

console.log(data);
// data berisi array program
```

3 Mapping & Integrasi

Data dari API diubah (mapping) ke format yang sesuai dengan UI Pekan 2.

```javascript
// Mapping data sesuai kebutuhan UI
const programs = data.map(item => ({
  id: item.id,
  title: item.name,
  description: item.description,
  image: item.image,
  status: item.status
}));

setPrograms(programs); // tampil di UI
```

4 Hasil Akhir (Dynamic Data)

UI yang sama dari Pekan 2 sekarang menampilkan data nyata dari API.



Alur Singkat

1. **PresidentKu UI Pekan 2** (sudah ada)
2. **Request** ke API
3. **REST API** Mengirim data program
4. **JSON** Data program
5. **Parse & Mapping** Ubah ke data model aplikasi
6. **Tampilkan di UI** Gunakan komponen yang sudah dibuat

Task

1. Membuat API request
2. Mengambil data dari API
3. Parsing JSON
4. Mapping data ke format UI
5. Menampilkan data pada UI yang sudah dibuat pada Pekan 2

Output

- **PresidentKu** menampilkan data program dari API secara dinamis
- Menggunakan interface yang sudah dibuat (sebelumnya)

Mobile Programming

DEMO – SHOW YOUR API INTEGRATION

Jalankan aplikasi dan tunjukkan proses pengambilan data dari API sampai tampil di UI.

Tujuan

Mahasiswa mendemonstrasikan hasil integrasi API pada aplikasi PresidentKu.

1 RUN

Jalankan aplikasi di Expo Go.



2 REQUEST

Aplikasi melakukan HTTP request ke API.

programService.js

```
const response =
  await fetch(
   "https://api.example.com/
    programs"
  );

const data =
  await response.json();
```

GET /programs

3 RESPONSE

Server mengirimkan data dalam format JSON.

Response | Headers | Preview

200 OK


```
[
  {
    "id": 1,
    "name": "Program Prioritas",
    "description": "Program
    strategis untuk ...",
    "image": "https://..../1.jpg",
    "status": "active"
  },
  {
    "id": 2,
    "name": "Program Pendidikan",
    "description": "Peningkatan
    kualitas pendidikan ...",
    "image": "https://..../2.jpg",
    "status": "active"
] }
```


4 PARSE

JSON diproses dan diubah ke data model (DTO).

mapping.js

```
const programs = data.map(
  item => ({
   id: item.id,
   title: item.name,
   description: item.description,
   image: item.image,
   status: item.status,
  })
);

setPrograms(programs);
// simpan ke state untuk UI
```

5 DISPLAY

Data ditampilkan pada UI PresidentKu yang sudah dibuat.



6 HASIL

PresidentKu sekarang menggunakan data nyata dari API (dynamic data).



Pertanyaan Demo dan Jawaban

1 Dari mana data aplikasi berasal?

Data berasal dari REST API yang disediakan oleh server, bukan lagi data statis di aplikasi.

2 Endpoint apa yang digunakan?

Menggunakan endpoint **https://api.example.com/programs** untuk mengambil data program.

3 HTTP method apa yang digunakan?

Menggunakan method **GET** untuk mengambil data.

4 Apa bentuk response API?

Response berupa data JSON (array of objects) yang berisi informasi program.

5 Bagaimana JSON diubah menjadi data yang digunakan UI?

JSON di-parse menjadi objek JavaScript, kemudian di-mapping ke data model/DTO yang sesuai dengan kebutuhan UI.

6 Mengapa data model/DTO diperlukan?

DTO diperlukan untuk memilih hanya field yang dibutuhkan, memastikan struktur data sesuai dengan aplikasi, dan memudahkan pengelolaan serta konsistensi data di seluruh komponen UI.

Mobile Programming

REVIEW – FROM STATIC TO DYNAMIC

Melihat kembali perkembangan aplikasi PresidentKu dari data statis menjadi data dinamis dari REST API.

1 CURRENT

PresidentKu (Static Data)

Pada pekan sebelumnya, data masih menggunakan data statis (dummy) yang disimpan langsung di dalam kode.



Contoh Static Data (Pekan 2)

```
// data langsung di dalam kode
const programs = [
  { id: 1, title: "Program Prioritas",
    description: "Program strategis ..." },
  { id: 2, title: "Program Pendidikan",
    description: "Peningkatan kualitas ..." }
];
```

2 REVIEW

Proses Integrasi API

Memahami alur lengkap mulai dari request ke API sampai data ditampilkan di UI.

- **API** — Server menyediakan data program
- **Request** — Aplikasi mengirim HTTP request
- **Response** — Server mengirim data dalam format JSON
- **JSON** — Data JSON diterima dan diparsing
- **Mapping** — Data diubah ke data model (DTO) sesuai kebutuhan UI

Contoh Response API (JSON)

```
[{
  "id": 1,
  "name": "Program Prioritas",
  "description": "Program strategis
    untuk meningkatkan kualitas ...",
  "image": "https://.../1.jpg",
  "status": "active"
}, ... ]
```

Mapping ke Data Model

```
const programs = data.map(item => ({
   id: item.id,
   title: item.name,
   description: item.description,
   image: item.image,
   status: item.status
}));
// digunakan untuk ditampilkan di UI
```

3 IMPROVED

PresidentKu (Dynamic Data)

Sekarang aplikasi mengambil data dari REST API sehingga data bisa berubah secara dinamis.



Review Checklist

Pastikan semua poin berikut sudah dipahami dan berjalan dengan baik.

- ✅ **Client-server dipahami** — Memahami alur komunikasi antara aplikasi dan server.
- ✅ **Endpoint benar** — Menggunakan endpoint yang sesuai.
- ✅ **HTTP method sesuai** — Menggunakan method yang tepat (GET untuk mengambil data).
- ✅ **Request berhasil** — Aplikasi berhasil mengirim request ke API.
- ✅ **Response diterima** — Server mengirim response dengan status code 200 OK.
- ✅ **JSON berhasil diparsing** — Data JSON berhasil diubah menjadi objek JavaScript.
- ✅ **Data model jelas** — Field yang dibutuhkan sudah ditentukan dan dimapping dengan benar.
- ✅ **Data API tampil pada UI** — Data dari API muncul di interface PresidentKu.
- ✅ **Tidak hard-code seluruh data** — Data diambil dari API, bukan lagi ditulis langsung di dalam kode.
- ✅ **UI Pekan 2 tetap digunakan** — Menggunakan interface yang sudah dibuat sebelumnya, hanya diganti sumber datanya.

Mobile Programming

SPRINT 03
CONNECT PRESIDENTKU TO REST API

Checklist penyelesaian sprint untuk memastikan semua materi dan task sudah dikuasai dengan baik.

Tujuan Sprint 03

Aplikasi PresidentKu berhasil mengambil data dari REST API dan menampilkannya pada mobile UI secara dinamis.



SPRINT 03 COMPLETION CHECKLIST

Pastikan semua item berikut sudah dipelajari dan berfungsi dengan baik.

1. ☐ **Client-server architecture** — Memahami alur komunikasi antara aplikasi (client) dan server (REST API).
2. ☐ **RESTful API** — Memahami konsep RESTful API dan cara kerjanya.
3. ☐ **Endpoint** — Mengetahui endpoint yang digunakan untuk mengambil data program.
4. ☐ **HTTP method** — Menggunakan HTTP method yang sesuai (GET untuk mengambil data).
5. ☐ **Request** — Membuat dan mengirim HTTP request dari aplikasi.
6. ☐ **Response** — Menerima response dari server dengan status code yang benar.
7. ☐ **JSON** — Memahami struktur data JSON dari response API.
8. ☐ **Data modeling** — Menentukan data apa saja yang dibutuhkan oleh aplikasi PresidentKu.
9. ☐ **DTO concept** — Membuat Data Transfer Object (DTO) untuk memetakan data API ke data aplikasi.
10. ☐ **Fetch / Axios** — Menggunakan Fetch API atau Axios untuk mengakses REST API.
11. ☐ **API integration** — Mengintegrasikan API ke dalam aplikasi PresidentKu.
12. ☐ **JSON parsing** — Memparsing JSON menjadi objek JavaScript / TypeScript.
13. ☐ **Data mapping** — Mengubah data API sesuai dengan format data model (DTO) untuk UI.
14. ☐ **Dynamic mobile UI** — Menampilkan data API pada UI PresidentKu yang sudah dibuat pada Pekan 2.
15. ☐ **Test pada Expo Go** — Menjalankan aplikasi di Expo Go dan memastikan data tampil dengan benar.
16. ☐ **Commit** — Melakukan commit kode perubahan ke repository Git.
17. ☐ **Push** — Melakukan push kode ke repository Git (GitHub/GitLab).
18. ☐ **Ready for next sprint** — Semua materi dan task sudah selesai dan siap melanjutkan ke Sprint berikutnya.



SPRINT 03 COMPLETED!

Hasil yang Diharapkan

- PresidentKu terhubung dengan REST API
- Data program tampil secara dinamis
- UI Pekan 2 tetap digunakan
- Kode sudah di-commit dan di-push
- Siap melanjutkan ke Sprint 04

NEXT SPRINT

Sprint 04 — Offline-First & Local Storage